



SAPIENZA
UNIVERSITÀ DI ROMA

Practical Network Defense a.y. 23-24

Assignment 2 Report - ACME 27 Group

Mija Simone Nicolae, 1895592 - mija.1895592@studenti.uniroma1.it

Palozzi Massimo, 2132174 - palozzi.2132174@studenti.uniroma1.it

Scarselli Ilaria, 1918975 - scarselli.1918975@studenti.uniroma1.it

Uffreduzzi Giorgia, 1668798 - uffreduzzi.1668798@studenti.uniroma1.it

October 23, 2024



Contents

1	Introduction	3
2	Road-warriors VPN	3
2.1	Accessing the Main Firewall Web Interface	3
2.2	Creating a Certificate Authority (CA)	3
2.3	Creating a Server Certificate	3
2.4	Configuring the OpenVPN Server	4
2.5	Configuring Firewall Rules	4
2.6	Creating Users and Assigning Certificates	5
2.7	User Group Configuration	5
2.8	Static IP addresses	6
2.9	Exporting Client Configuration	7
2.10	Configuring the OpenVPN Client	7
2.11	Tests	7
3	Main-Internal IPsec tunnel	8
3.1	Phase 1	8
3.2	Phase 2	8
3.2.1	Main Firewall	8
3.2.2	Internal Firewall	9
3.3	Final adjustments and testing	9
4	Conclusions	10

1 Introduction

This report outlines the configuration and implementation of VPN services for ACME network. The assignment involved setting up two VPNs: one for the road-warriors remote employees using the Main-firewall as a VPN gateway, and another for secure communication between the Main and Internal routers via an IPSec tunnel. Both VPNs were configured to work with IPv4 and IPv6, and the detailed steps taken are detailed in the following sections.

2 Road-warriors VPN

For this first part, we decided to implement OpenVPN on OPNsense because is widely recognized for its robust security features and flexibility, making it an ideal choice for creating secure connections for remote users.

2.1 Accessing the Main Firewall Web Interface

To begin the configuration process, we accessed the Main Firewall web interface by opening a web browser and navigating to the IP address of the OPNsense device at 100.100.6.1.

2.2 Creating a Certificate Authority (CA)

Once logged in, we created a Certificate Authority (CA), which was essential for issuing certificates that authenticate both the OpenVPN server and its clients. We navigated to System → Trust → Authorities and clicked on Add. Here, we filled out the form, providing a descriptive name such as "ACME OPNVPN CA", selected the method to create an internal CA, and defined the key length, choosing 2048 bits for enhanced security. After setting the desired certificate validity period, we clicked Save to create the CA.

System: Trust: Authorities				
Name	Internal	Issuer	Certificates	Distinguished Name
ACME OPNVPN CA	YES	self-signed	4	emailAddress=admin@acme-27.test, ST=Rome, O=ACME, L=Rome, CN=acme_opnvpn_ca, C=IT
			Valid From:	Mon, 14 Oct 2024 21:09:39 +0000
			Valid Until:	Sun, 17 Jan 2027 21:09:39 +0000

2.3 Creating a Server Certificate

With the CA established, our next task was to create a server certificate. This certificate was necessary as it enabled the OpenVPN server to authenticate itself to clients. We headed to System → Trust → Certificates, clicked on Add, and completed the required fields. We selected "ACME OPNVPN CA"

as the certificate authority, chose "Server Certificate" as the key type, and set the validity period and common name, we finalized this step by clicking Save.

2.4 Configuring the OpenVPN Server

Once the necessary certificates were in place, we configured the OpenVPN server itself. We navigated to VPN → OpenVPN and selected the Servers tab, where, clicking on Add, we started a new configuration. Here, we specified various settings. Selected the authentication backend, UDP protocol, specified the "WAN" interface and added the local port 1194. We also defined the Server (IPv4) that is the virtual subnet that the VPN would create. We did the same for IPv6. Finally, clicking on Save, the new server have been added.

General Settings	
Role	Server
Description	RoadWarrior OpenVPN
Enabled	☒
Protocol	UDP
Port number	1194
Bind address	
Server (IPv4)	100.100.253.0/24
Server (IPv6)	2001:470:b5b8:1bfd::/64
Topology	subnet
Trust	
Certificate	ACME OPNVPN CERTIFICATE
Certificate Revocation List	Nothing selected
Verify Client Certificate	required
Use OCSP (when available)	☐
Certificate Depth	Do Not Check
TLS static key	None

2.5 Configuring Firewall Rules

To allow external connections to the OpenVPN server, it was essential to configure the necessary firewall rules. This was done by navigating to Firewall → Rules and selecting the WAN interface. Here, we created a new rule by clicking Add. This rule specified that the action was to "Pass," allowing traffic through the WAN interface. We configured the protocol as UDP, set the source to "Any," and directed the destination to the WAN address on the specified port. After saving and



applying these changes, RoadWarriors were able to connect to the OpenVPN server.

Firewall: Rules: OpenVPN

	Protocol	Source	Port	Destination	Port	Gateway	Schedule	Description	
Automatically generated rules									
☐	IPv4-6 UDP	*	*	*	1194 (OpenVPN)	*	*	Allow opnvpn traffic	
☐	IPv4 *	Operators_ipv4	*	*	*	*	*	Allow operators traffic	
☐	IPv6 *	Operators_ipv6	*	*	*	*	*	Allow operators traffic	
☐	IPv4 *	Employees_ipv4	*	*	*	*	*	Allow employees traffic	
☐	IPv6 *	Employees_ipv6	*	*	*	*	*	Allow employees traffic	
	pass	block (disabled)		reject		log		in	first match
	block (disabled)		reject		log (disabled)			out	last match

2.6 Creating Users and Assigning Certificates

With the server and firewall configured, our next step was to create user accounts and assign them certificates. We navigated to System → Access → Users and clicked on Add. There, we created an internal user certificates, giving it an identifiable name. Once saved, these user accounts were ready for authentication when connecting to the VPN.

System: Trust: Certificates

Name	Issuer	Distinguished Name	
Web GUI SSL certificate	self-signed	ST=Zuid-Holland, O=OPNsense, L=Middelharnis, C=NL Valid From: Mon, 10 Feb 2020 10:38:37 +0000 Valid Until: Sun, 15 May 2022 10:38:37 +0000 CA: No, Server: Yes	
ACME OPNVPN CERTIFICATE	ACME OPNVPN CA	emailAddress=admin@acme-27.test, ST=Rome, O=ACME, L=Rome, CN=acme_opnvpn_certificate, C=IT Valid From: Mon, 14 Oct 2024 21:13:59 +0000 Valid Until: Sat, 15 Nov 2025 21:13:59 +0000 CA: No, Server: Yes	
Alice	ACME OPNVPN CA	emailAddress=admin@acme-27.test, ST=Rome, O=ACME, L=Rome, CN=Alice, C=IT Valid From: Mon, 14 Oct 2024 21:25:26 +0000 Valid Until: Sat, 15 Nov 2025 21:25:26 +0000 CA: No, Server: No	User Cert
Bob	ACME OPNVPN CA	emailAddress=admin@acme-27.test, ST=Rome, O=ACME, L=Rome, CN=Bob, C=IT Valid From: Mon, 14 Oct 2024 21:28:52 +0000 Valid Until: Sat, 15 Nov 2025 21:28:52 +0000 CA: No, Server: No	User Cert
Charles	ACME OPNVPN CA	emailAddress=admin@acme-27.test, ST=Rome, O=ACME, L=Rome, CN=Charles, C=IT Valid From: Mon, 14 Oct 2024 21:30:43 +0000 Valid Until: Sat, 15 Nov 2025 21:30:43 +0000 CA: No, Server: No	User Cert

2.7 User Group Configuration

To configure user access within the ACME Co. network, we accessed the group management section in OPNsense. First, we went to System → Access → Groups. There, we created a new group named "Operators" for the operators, which had full access to all internal networks. Next, we created a second group called "Employees," designated for employees, who had limited access to specific networks only.

System: Access: Groups

Group name	Member Count	Description	
 admins	1	System Administrators	
 employees	2	can access all the networks of ACME except internal server network.	 
 operators	1	can access all the networks of ACME	 

 Superuser group
  Normal group

After creating the groups, we proceeded to create the users. We returned to System → Access → Users. We started by adding the first user, Alice, specifying her username. For Alice, we selected the "Operators" group, ensuring she had access to all ACME networks. Next, we created Bob by entering his username, but this time we associated him with the "Employees" group. This way, Bob had access only to the specified networks, because from the requirements he don't have access to the server network. Finally, we repeated this process for Charles, making sure to assign him to the "Employees" group, granting him the same access rights as Bob.

Here passwords for the users authentication:

Alice: *Y a27@.wVgp_ : XL?*

Bob: *a321..LwVgfgKdL!*

Charles: *N358!GLNNsfvvdL.*

System: Access: Users

Username	Full name	Groups	
 Alice	Alice	operators	 
 Bob	Bob	employees	 
 Charles	Charles	employees	 
 root	System Administrator	admins	

 System Administrator
  Disabled User
  Normal User

Once the users and groups were configured, we set up firewall rules to ensure that access rights were correctly enforced. For Alice, the rules allowed access to all the networks, while for Bob and Charles, we created rules that restricted access only to the networks they were authorized to enter.

2.8 Static IP addresses

The VPN server assigned IP addresses to its hosts dynamically by choosing from the available addresses within the VPN subnet pool. This approach made it challenging to manage access within the ACME network. To resolve this issue, we opted to assign a specific static IP address to each

client. In OPNsense, we accessed the section for VPN $\rightarrow$ OpenVPN $\rightarrow$ Client Specific Overrides and created three rules, one for each road warrior.

VPN: OpenVPN: Client Specific Overrides

Overwrites

Enabled	Common name	Tunnel Network	Description	Commands
☒	Alice	100.100.253.2/32 2001:470:b5b6:1b4d::20/124	RoadWarrior Alice Accessing	Edit Delete
☒	Bob	100.100.253.6/32 2001:470:b5b6:1b4d::60/124	RoadWarrior Bob Accessing	Edit Delete
☒	Charles	100.100.253.10/32 2001:470:b5b6:1b4d::100/124	RoadWarrior Charles Accessing	Edit Delete

This configuration enabled us to assign the necessary permissions for each role by creating aliases in the main firewall. We proceeded to Firewall → Aliases and established the alias “Operators,” which contained the IPv4 address designated for Alice, and “Employees,” which included the IPv4 addresses for Bob and Charles.

2.9 Exporting Client Configuration

To facilitate client connections, we exported the client configuration. This was done by navigating to VPN $\rightarrow$ OpenVPN $\rightarrow$ Client Export. There, we selected the client we had just created from the list and chose the desired export format. By clicking Download, we saved the configuration file necessary for the client setup.

2.10 Configuring the OpenVPN Client

Now users can connect to the VPN by selecting the imported profile and entering their credentials when prompted.

2.11 Tests

To ensure that the VPN is functioning correctly, we connected to the VPN tunnel as Alice from our virtual machine. This confirmed that the VPN setup was working.

Time	Source	Destination	Protocol	Length	Info
20.752991	10.0.0.253	10.0.0.2	HTTP	135	GET / HTTP/1.1
20.773230	10.0.0.253	10.0.0.2	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
20.773378	10.0.0.2	10.0.1.4	OpenVPN	56	Message-Type: P.DATA_V2
20.773595	10.0.0.2	10.0.1.4	OpenVPN	104	564 HTTP/1.1 → 563 Moved Permanently (text/html)
32.733759	10.0.0.2	10.0.1.4	OpenVPN	626	Message-Type: P.DATA_V2
53.750059	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.741504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) request id=0x000a, seq=2512, ttl=64 (request in 47)
47.539727	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=2512, ttl=62 (request in 46)
48.539922	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
49.539550	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
50.539993	10.0.0.2	10.0.1.2	ICMP	88	echo (ping) request id=0x000a, seq=3768, ttl=64 (reply in 51)
51.540400	10.0.1.2	10.0.0.2	ICMP	88	echo (ping) reply id=0x000a, seq=3768, ttl=62 (request in 50)
52.540599	10.0.0.2	10.0.1.4	OpenVPN	110	Message-Type: P.DATA_V2
53.541159	10.0.0.2	10.0.1.4	TCP	56	56 → 41966 [ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
54.541504	10.0.1.4	10.0.0.2	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
56.844851	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
56.844874	10.0.0.2	10.0.0.2	TCP	56	41966 → 88 [FIN, ACK] Seq=475 Win=31872 Len=0 TSval=330814722 TSecr=530814722
37.844118	10.0.0.2	10.0.0.253	TCP	56	56 → 41966 [FIN, ACK] Seq=475 Win=6448 Len=0 TSval=330814722 TSecr=389588587
38.844118	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
39.859222	10.0.1.4	10.0.0.2	TCP	110	Message-Type: P.DATA_V2
39.859275	10.0.0.2	10.0.1.4	TCP	56	41966 → 56 [ACK] Seq=475 Win=31872 Len=0 TSval=330859856 TSecr=530814722
41.535994	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
42.536145	10.0.0.2	10.0.0.2	ICMP	88	echo (ping) request id=0x000a, seq=1256, ttl=64 (request in 43)
43.537135	10.0.0.2	10.0.0.253	ICMP	88	echo (ping) reply id=0x000a, seq=1256, ttl=62 (request in 42)
43.537588	10.0.0.2	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
45.538227	10.0.1.4	10.0.0.2	OpenVPN	110	Message-Type: P.DATA_V2
46.538384	10.0.0.2	10.0.0.253	ICMP	88	

We successfully pinged the DNS server through our tun0 interface and made http request to the Webserver in DMZ.

3 Main-Internal IPsec tunnel

For the second part of the assignment, all traffic must pass through a site-to-site IPsec tunnel. We followed the OPNsense documentation, especially for Phase 1, and then adapted the configuration to meet our specific needs. Both phases, as well as the subsequent adjustments to the pre-existing configurations, must be implemented on both the Main and Internal Firewalls. We configured everything to work with both IPv4 and IPv6 traffic.

3.1 Phase 1

Before starting, we added the following rules to the INTERNAL interface of the Main Firewall and the EXTERNAL interface of the Internal Firewall, to allow IPsec tunnel connections:

☐		IPv4+6 ESP	*	*	*	*	*	*	Allow ipsec tunneling
☐		IPv4+6 TCP/UDP	*	500 (ISAKMP)	*	*	*	*	Allow ipsec tunneling
☐		IPv4+6 TCP/UDP	*	4500 (IPsec NAT-T)	*	*	*	*	Allow ipsec tunneling

Then, under VPN > IPsec > Tunnel Settings tab of the Main Firewall, we added one entry for IPv4 traffic with the following steps and options:

- Our local interface is the INTERNAL;
- The remote gateway is *100.100.254.2*, that is the one of the EXTERNAL interface of the other router;
- Selected Mutual PSK for the authentication method, leaving the default "My / Peer IP address" as identifiers for simplicity;
- Generated a Pre-Shared Key in base64
- Selected the encryption parameters (256 bit AES-GCM with 128 bit ICV, SHA512 and Diffie-Hellman key group 14 [2048 bits]) to achieve a suitable tradeoff between performance and security, with a robust hashing algorithm to protect against collisions.

We applied this configuration for IPv6 as well (by adding a new entry) and repeated it for the Internal Firewall as well, adjusting the IP addresses and interfaces accordingly.

3.2 Phase 2

3.2.1 Main Firewall

The entries for phase two were the following:



☐ Enabled	ReqId	Type	Local Subnet	Remote Subnet	Phase 2 Proposal	Description
☒	13	ESP IPsec tunnel	0.0.0.0	100.100.10.24	any255gtr010 + SHA512+ DH Group 14	tunnel IPsec to internal servers
☒	14	ESP IPsec tunnel	0.0.0.0	100.100.2.24	any255gtr010 + SHA512+ DH Group 14	tunnel IPsec to clients network

Phase 2						Q	Search
☐ Enabled	ReqId	Type	Local Subnet	Remote Subnet	Phase 2 Proposal	Description	
☒	15	ESP IPsec tunnel	::0	2001:470:5008:2002::04	any255gtr01 + SHA512+ DH Group 14	tunnel IPsec to clients network	
☒	16	ESP IPsec tunnel	::0	2001:470:5008:2002::04	any255gtr01 + SHA512+ DH Group 14	tunnel IPsec to servers network	

where we used the address 0.0.0.0/0 (or ::/0 for IPv6) as the local network address to allow all traffic, even the one from the internet, to reach the Internal Firewall subnets through the tunnel, and we chose the ESP protocol to encapsulate and encrypt the packets with options similar to those used in Phase 1 regarding the algorithms employed.

Then, we checked the "Enable IPsec" box.

3.2.2 Internal Firewall

We followed roughly the same steps as the ones for the Main Firewall, but this time we used the 0.0.0.0/0 (and ::/0 for IPv6) address in the remote network field to allow traffic from the Internal Firewall subnets to flow through the tunnel and into any other subnet of the Main, as well as the Internet:

Phase 2					Search	
☐ Enabled	ReqId	Type	Local Subnet	Remote Subnet	Phase 2 Proposal	Description
☒ 13	ESP IPsec tunnel	CLIENTS	0.0.0.0/0	0.0.0.0/0	any255gtr010 + SHA512+ DH Group 14	tunnel ipsec clients network to internet
☒ 14	ESP IPsec tunnel	SERVICES	0.0.0.0/0	0.0.0.0/0	any255gtr010 + SHA512+ DH Group 14	tunnel ipsec servers network to internet

Phase 2

☐

Enabled

☒

16

☒

17

ReqId

Type

Local Subnet

Remote Subnet

Phase 2 Proposal

Description

ESP IPsec tunnel

2001:470:5008:2002::04

::0

any255gtr010 + SHA512+ DH Group 14

tunnel IPsec internal servers to internet

ESP IPsec tunnel

2001:470:5008:2002::04

::0

any255gtr010 + SHA512+ DH Group 14

tunnel IPsec internal clients to internet

Additionally, on this tunnel endpoint, we also enabled the **Passthrough network** option under VPN > IPsec > Advanced Settings specifying the IPv4 and IPv6 subnet prefixes of the Internal Servers Network and the Clients Network to avoid forcing the two local LANs through the tunnel. This step was not necessary on the Main Firewall.

3.3 Final adjustments and testing

At the end of everything, we disabled all the rules (except for the ones needed for the IPsec traffic to flow) on the firewall INTERNAL and EXTERNAL interfaces and moved them to the newly created IPsec interface, in order to maintain the desired traffic restrictions through the new tunnel. To test this part, we performed a tcpdump and checked that all the packets were correctly encapsulated and passed through the tunnel:

14	0.873870	100.100.254.2	100.100.254.1	ESP	190	ESP (SPI=0xc729ffba)
15	0.874516	100.100.254.1	100.100.254.2	ESP	190	ESP (SPI=0xcc6e9c59)
16	0.874571	100.100.254.2	100.100.254.1	ESP	190	ESP (SPI=0xc729ffba)
17	0.874857	100.100.254.1	100.100.254.2	ESP	190	ESP (SPI=0xcc6e9c59)
18	0.874910	100.100.254.2	100.100.254.1	ESP	190	ESP (SPI=0xc729ffba)
19	0.875170	100.100.254.1	100.100.254.2	ESP	190	ESP (SPI=0xcc6e9c59)
20	0.875219	100.100.254.2	100.100.254.1	ESP	190	ESP (SPI=0xc729ffba)
21	0.875507	100.100.254.1	100.100.254.2	ESP	190	ESP (SPI=0xcc6e9c59)
22	0.875554	100.100.254.2	100.100.254.1	ESP	190	ESP (SPI=0xc729ffba)



4 Conclusions

We successfully configured VPN services for the ACME network, ensuring secure communication for both remote users and inter-site connections. The road warriors VPN was implemented to provide access for employees and operators, with the appropriate access restrictions enforced. Additionally, the IPSec tunnel between the Main and Internal routers secured traffic across an otherwise insecure medium. Throughout the process, we tested both IPv4 and IPv6 configurations, ensuring proper connectivity and security.